QueryAPIGate HOW-TO GUIDE 56 · ACCESS

Use short-lived database credentials from Vault

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=56-use-short-lived-database-credentials-from-vault

Time: 20 minutes. **You'll end up with:** QueryAPIGate with no database password anywhere - Vault makes a database user for each lease, QueryAPIGate moves to the next one before the last expires, and not one call fails while it does; when Vault is down, it keeps serving while it safely can, then fails closed.

> **Stable** - secrets from Vault, and its short-lived database credentials, are under the compatibility promise since 0.34 (what that means). AWS's are still experimental.

Every command and output below was run against **Vault 1.21** (dev mode) and **PostgreSQL 16**, with QueryAPIGate signing in to Vault through **AppRole**. The leases last **60 seconds** so that a few rotations fit in the guide; an hour is usual.

Step 1: Vault makes the database's users

What Vault's administrator sets up - the database secrets engine, managing a role that may only read `revenue`:

shell

```
vault secrets enable database
vault write database/config/warehouse plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@warehouse.internal:5432/postgres" \
  username=vault password=... allowed_roles=reporting-ro
vault write database/roles/reporting-ro db_name=warehouse default_ttl=60s max_ttl=1h \
  creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL
'{{expiration}}';
                       GRANT SELECT ON revenue TO \"{{name}}\";"
```

And for QueryAPIGate, a policy that reaches exactly that - and revoking its own leases - with an AppRole for it to sign in with:

hcl

```
path "database/creds/reporting-ro" { capabilities = ["read"] }
path "sys/leases/revoke"          { capabilities = ["update"] }
```

shell

```
vault auth enable approle
vault write auth/approle/role/queryapigate token_policies=queryapigate token_ttl=1h
vault read auth/approle/role/queryapigate/role-id         # the role ID
vault write -f auth/approle/role/queryapigate/secret-id   # the secret ID - into a file only
QueryAPIGate reads
```

Step 2: QueryAPIGate, pointed at Vault

```shell
QUERYAPIGATE_VAULT_ADDR=https://vault.example.com:8200
QUERYAPIGATE_VAULT_AUTH=approle
QUERYAPIGATE_VAULT_ROLE_ID=<the role ID>
QUERYAPIGATE_VAULT_SECRET_ID_FILE=/run/secrets/vault-secret-id
QUERYAPIGATE_VAULT_DATABASE_ROLES=reporting-ro@warehouse.internal:5432
```

The last line says which roles a connection may ask for - **and where their credentials may go**: `reporting-ro` only to the warehouse. (On Kubernetes, `QUERYAPIGATE_VAULT_AUTH=kubernetes` with the chart's `vaultToken.enabled`, and no secret ID at all.)

Step 3: A connection with no password

```json
POST /api/v1/connections
{"name": "warehouse", "db": "postgres", "host": "warehouse.internal", "port": 5432, "database":
"postgres",
 "credentials": {"source": "vault-database", "role": "reporting-ro"}}
```

A saved query on it, `revenue`, that also says who it ran as:

```sql
SELECT current_user AS connected_as, month, amount FROM revenue ORDER BY month
```

```text
GET /q/revenue
[{"connected_as": "v-approle-reportin-8zBMUctzHI5CG3EW6P4s-1791811633", "month": "2026-08", "amount":
98000},
 {"connected_as": "v-approle-reportin-8zBMUctzHI5CG3EW6P4s-1791811633", "month": "2026-09", "amount":
104500}]
```

A user Vault made a moment ago, for this lease. Pointing the role anywhere else is refused - otherwise an administrator could have the server log in to a host of theirs with the warehouse's credentials:

```json
POST /api/v1/connections
{"name": "elsewhere", "host": "db.attacker.example", ..., "credentials": {"source": "vault-database",
"role": "reporting-ro"}}

400 {"code": "invalid_body", "error": "credentials vault-database is refused: 'reporting-ro' is bound to
     warehouse.internal:5432 by QUERYAPIGATE_VAULT_DATABASE_ROLES - not db.attacker.example:5432"}
```

Step 4: Leases come and go; calls don't fail

`GET /q/revenue` every 200 ms for three minutes - three of those 60-second leases:

```text
answered 861, failed 0
  from 18:57:24: v-approle-reportin-8zBMUctzHI5CG3EW6P4s-1791811633 (179 calls)
  from 18:58:01: v-approle-reportin-a4lOjbOTU2pSuW4ykWGG-1791811681 (229 calls)
  from 18:58:49: v-approle-reportin-WwAVdOPwmARxZO2aH8m4-1791811729 (231 calls)
  from 18:59:38: v-approle-reportin-1dRYtNa6w8ziCcjnsNmr-1791811778 (222 calls)
```

Four database users, each taking over about 48 seconds after the last: QueryAPIGate asks Vault for the next pair at 80% of a lease, and stops using a pair - and retires every pooled connection opened with it - a tenth of the lease before it ends (at least 5 seconds, at most 5 minutes). Every connection opened in between shares one pair: one lease in Vault, not one per connection.

Step 5: Vault goes down

Vault paused right after QueryAPIGate took a fresh pair, calls once a second:

```text
19:06:02  Vault paused
19:06:02  answered as v-approle-reportin-HCkAeIlkRnhgyui...
19:07:01  answered as v-approle-reportin-HCkAeIlkRnhgyui...   (10.1s)
19:07:02  503 secret_unavailable
19:07:21  503 secret_unavailable                              (10.1s)
19:07:23  Vault back
19:07:34  answered as v-approle-reportin-qYgPOHrZaN6Bsso...
```

- For 48 seconds nothing changed: the pair it held was good.
- At the refresh point, one call waited on Vault - the 10-second timeout - and was still answered with the pair still within its time. The log says so, as a warning: *The credential source vault-database failed (Vault at … could not be reached: timed out) - using the credentials it gave before, good for …s more*.
- Then the pair was too close to its end, and QueryAPIGate **failed closed**: `503 secret_unavailable`, at once - Vault is asked again only every 10 seconds, so calls don't each wait on it.
- With Vault back, the next call got a new pair.

After five minutes of that, the `secret_provider_failing` alert says so, and Settings shows each provider's last answer: `vault-database: ok at 19:01:55`, or `failing since ...` with the error.

Step 6: Nothing left behind

Leases this process took are revoked when it stops - on `SIGTERM`, as Kubernetes and Docker send:

```text
leases while serving:  1
leases after SIGTERM:  0
```

So restarts don't fill Vault with leases waiting out their TTL.

Good to know

- **A password in Vault's KV store** is `${vault:secret/data/warehouse#password}` in any connection field, read under `QUERYAPIGATE_VAULT_PATHS` only - for a database that can't have users made per lease.
- **RDS and Aurora** need no Vault: `"credentials": {"source": "aws-rds-iam"}` makes an IAM token the password, for hosts in `QUERYAPIGATE_AWS_RDS_HOSTS`, over TLS only.
- **The lease length is Vault's.** An hour is usual; QueryAPIGate then takes a new pair every 48 minutes, and Vault being down is invisible for up to 54.
- **Pooled connections** are retired before their pair's lease ends, however recently used - a query never runs on a user Vault is about to drop.

Next steps

- Secrets from files and secrets managers, in the reference.
- Install on Kubernetes with Helm - `vaultToken` for Kubernetes auth.
- Read the threat model.